Instant Messaging service using RESTful Web Services and WebSockets

Juan Luis Gorricho

Second approach

- This is a second approach implementing an instant messaging service, a remote approach without persistence.
- This approach focuses on using a web application service and on accessing the service using a REST API.
- The approach uses Swing clients instead of Internet browsers, and the **asynchronous** behavior will be implemented with a pushing strategy using a WebSockets library. The client has no persistence either.

Exercise

- Open the **InstantMessagingRemote_server** web application project. Import the GSON library from the libs folder if necessary.
- Add the code from the previous exercise implementing: **PublisherImpl** and **TopicManagerImpl**, be careful not to delete some extra methods.
- Run the web application project.

Exercise

- Try to understand the meaning of all the code contained in the web app. project.
- In particular, review:
  - The two **FacadeREST** included in the facadeREST package. Review the syntax specifying the URL paths to be used to execute the different methods. Pay attention to the use of an instance of **Global**, with application scope, to retrieve the instance of **TopicManager**.

Exercise

- In particular, review:
  - The programming of **WebSocketServer** where the server only accepts subscription requests from its clients to subscribe or unsubscribe that client to a topic.
  - The programming of **SubscriberImpl** where we use a WebSocket session to asynchronously send messages to the client.

Exercise

- Open the InstantMessagingRemote_client_exercise.
- Try to understand the coding of the Java classes included in apiREST as an example of the usual programming of a REST API. Notice all methods are programmed as **static** methods.
- Copy and paste the code of the SwingClient Java class from previous version.
- Complete the code of PublisherStub and TopicManagerStub, basically, to execute remote calls using the apiREST; except for the subscribe() and unsubscribe() methods where you have to use the WebSocketClient methods, why?

Exercise

- Complete the code of the WebSocketClient in accordance with the programming of the WebSocketServer and the SubscriberImpl at the server side.
- Review the schema of classes of the following slide for a better understanding of the exercise.
- Run separately two clients with different user names to test the code.

Schema of classes for a remote service

